Practica 4 Gestión de imágenes y contenedores

Nombre

Juan Manuel Rodriguez

Materia

Sistemas Distribuidos

Docente

Leidy Mariani Dorado

Corporación universitaria Unicomfacauca
Popayán-Cauca

PARTE 1 – Exploración de imágenes

1. Ejecutando “docker images”

   Actividad 1: ver imágenes locales

```
PS C:\Users\manur\Escritorio\compose-1> docker images

IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
api-node-demo:latest     6c2bdfe8ce7e        140MB             0B    U
mi_sitio_web:latest      004f13769b49        161MB             0B    U
mysql:latest             8458fdfc578c        922MB             0B    U
nginx:latest             fd204fe2f750        161MB             0B    U
portafolio:latest        004f13769b49        161MB             0B    U
sitio_web2:latest        004f13769b49        161MB             0B    U
```

- ¿Qué es el IMAGE ID?
  Es el identificador único de una imagen de Docker.
- ¿Qué significa TAG?
  Es una Marca alfanumérica asignada a una imagen Docker que funciona como un alias o identificador de versión.
- ¿Qué representa SIZE?
  Es la cantidad de espacio en disco que ocupan las imágenes y los contenedores.

2. **Actividad 2: Descargar nuevas imágenes**

Ejecutando “docker images” nuevamente

```
PS C:\Users\manur\Escritorio\compose-1> docker images

IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
api-node-demo:latest     6c2bdfe8ce7e        140MB             0B    U
httpd:latest             10e58ed1b203        117MB             0B
mi_sitio_web:latest      004f13769b49        161MB             0B    U
```

- ¿Qué cambio?
  Cambio la lista de imágenes que tenemos en docker, ahora tenemos la nueva imagen que es la de httpd.
- ¿El servicio ya está funcionado?

No, el servicio no esta funcionado porque reciente mente solo se descargo la imagen mas no se ha ejecutado.

PARTE 2- Gestión de contenedores

Actividad 3: Crear un contenedor

Para esta parte yo ya descargué la imagen de httpd entonces lo hare con esa imagen

- En la anterior imagen observamos que el contenedor httpd se creó correctamente

```
PS C:\Users\manur\Escritorio\compose-1> docker run -d --name servidor-httpd -p 8081:80 httpd
99a406fe8fb5f19f12887db5002a5ab3118b0006795d5480d13b3802ccac0716
```

- El contenedor se llama “servidor-httpd”
- Está corriendo en el puerto 8081

Con “docker ps” verificamos los contenedores que se están ejecutando, en nuestro caso aparece el contenedor httpd con sus características

```
PS C:\Users\manur\Escritorio\compose-1> docker ps
CONTAINER ID   IMAGE     COMMAND              CREATED             STATUS              PORTS
       NAMES
99a406fe8fb5   httpd     "httpd-foreground"   About a minute ago   Up About a minute   0.0.0.0:8081->80/tcp, [::]:8081->80
/tcp   servidor-httpd
```

- Abriendo el navegador en http://localhost:8081/ aparece lo siguiente:
  Esto quiere decir que nuestro contenedor httpd esta funcionando correctamente y el navegador pudo conectarse por http

It works!

Actividad 4: Administrar el contenedor

Detener

```
PS C:\Users\manur\Escritorio\compose-1> docker stop servidor-httpd
servidor-httpd
PS C:\Users\manur\Escritorio\compose-1> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS   PORTS   NAMES
PS C:\Users\manur\Escritorio\compose-1>
```

Ver todos los contenedores

```
PS C:\Users\manur\Escritorio\compose-1> docker ps -a
CONTAINER ID   IMAGE                 COMMAND                  CREATED        STATUS                        PORTS
            NAMES
99a406fe8fb5   httpd                 "httpd-foreground"       23 hours ago   Exited (0) About a minute ago
            servidor-httpd
e2461d6e10a9   api-node-demo         "docker-entrypoint.s…"   27 hours ago   Exited (255) 24 hours ago     0.0.0.0:3000
>3000/tcp   magical_goldberg
49411a2f2954   mi_sitio_web:latest   "/docker-entrypoint.…"   2 days ago     Exited (255) 29 hours ago     80/tcp
            tender_keldysh
b333ef4063bd   mysql                 "docker-entrypoint.s…"   2 days ago     Exited (137) 2 days ago
            compose-1-db-1
4072601011cd   nginx                 "/docker-entrypoint.…"   2 days ago     Exited (0) 2 days ago
            compose-1-web-1
PS C:\Users\manur\Escritorio\compose-1>
```

Reiniciar

```
PS C:\Users\manur\Escritorio\compose-1> docker start servidor-httpd
servidor-httpd
```

Eliminar

```
PS C:\Users\manur\Escritorio\compose-1> docker start servidor-httpd
servidor-httpd
```

```
PS C:\Users\manur\Escritorio\compose-1> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS   PORTS   NAMES
```

PARTE 3- Análisis conceptual

Concepto	Imagen	Contendor
¿Es ejecutable?	No se ejecuta sola; es como la “base” o plantilla.	Sí; es el programa ya corriendo (o listo para arrancar)
¿Ocupa recursos en ejecución?	No usa CPU ni memoria; solo ocupa espacio guardada.	Sí; cuando está encendido usa memoria, procesador y red.
¿Puede modificarse?	No por lo que son inmutables, por ello no se pueden modificar directamente, para cambiarla toca crear una nueva imagen basándose en la anterior	**Sí; puede cambiar mientras se usa, pero esos cambios se pueden perder si se borran.**

	mediante “dockerfile” o “docker commit”	
¿Puede eliminarse?	Si	Si

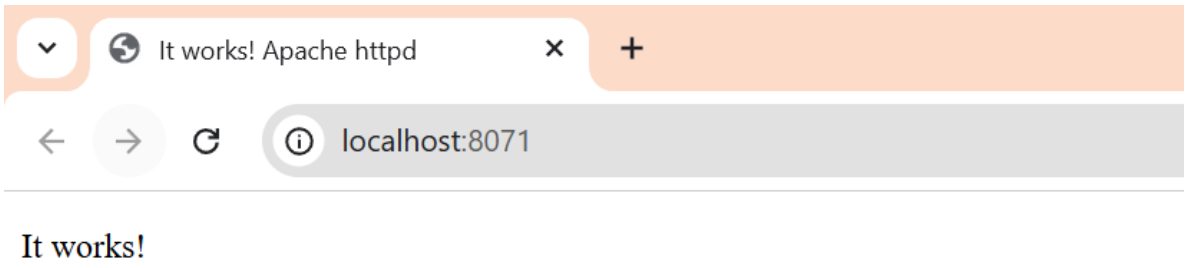
PARTE 4- Reto en equipo

1. Para el reto en equipo se utilizó “httpd” y se ejecuto

```
PS C:\Users\manur\Escritorio\compose-1> docker run -d --name servidor-citas -p 8071:80 httpd
e775c2c95e612996332e0c2281613e93b8be4794cdf814f2626be4b1bbef0988
```

2. Mostrando el servidor activo

```
PS C:\Users\manur\Escritorio\compose-1> docker ps
CONTAINER ID   IMAGE     COMMAND              CREATED         STATUS         PORTS                                       NAM
ES
e775c2c95e61   httpd     "httpd-foreground"   5 minutes ago   Up 5 minutes   0.0.0.0:8071->80/tcp, [::]:8071->80/tcp   ser
vidor-citas
```



It works!

3.

- ¿Qué tipo de servicio es?
  Es un servidor web Apache (HTTPD). Sirve para mostrar páginas web y entregar archivos o contenido web cuando alguien entra desde el navegador

- ¿En qué tipo de sistema distribuido podría usarse?
  Se puede usar en un sistema distribuido como una aplicación web con varias partes (por ejemplo, frontend + backend + base de datos), o en una arquitectura con varios servidores web detrás de un balanceador de carga para atender muchos usuarios al mismo tiempo.

- ¿Qué pasaría si ese servicio falla?
  Si falla, los usuarios no podrían acceder a la página o verían error (como “no responde” o “sitio caído”). Si hubiera varias copias en un sistema distribuido, el tráfico podría pasar a otra copia, pero si solo existe ese contenedor, el servicio quedaría totalmente caído hasta reiniciarlo.

PARTE 5-Analisis critico

1. ¿Cuál es la diferencia real entre imagen y contenedor?

   La diferencia fundamental es que una imagen es una plantilla estática y de solo lectura donde está el código y las dependencias, mientras que un contenedor es una instancia de ejecución de una imagen Docker.

2. ¿Por qué Docker facilita la escalabilidad?

   Docker facilita la escalabilidad porque nos permite crear y levantar copias de una aplicación muy rápido y de forma igual en cualquier lugar. Cuando todo viene empaquetado en contenedores, se puede aumentar la cantidad de contenedores cuando sube el trafico y cuando baja, sin tener que instalar ni configurar el programa una y otra vez. Eso da flexibilidad y velocidad para adaptarse a la demanda y probar cosas nuevas

3. ¿Cómo se relaciona esto con microservicios?

   En el enfoque de microservicios, Docker se usa para crear, desplegar y administrar las aplicaciones. Permite empaquetar cada microservicio en un contenedor independiente, de modo que se puedan ejecutar y actualizar por separado. Esto hace el despliegue más sencillo y reduce la posibilidad de problemas por dependencias, conflictos entre servicios y errores de configuración.

4. ¿Qué ventajas ofrece frente a instalar software directamente?

   Docker ofrece ventajas superiores a la instalación directa de software, destacando la portabilidad, el aislamiento y la consistencia entre entornos. Al

empaquetar aplicaciones con sus dependencias en contenedores ligeros, Docker elimina conflictos de versiones, facilita despliegues rápidos y optimiza el uso de recursos al compartir el núcleo del sistema operativo

Conclusiones

En esta guía aprendimos a diferenciar entre una imagen y un contenedor donde entendimos que la imagen es la base y el contenedor es el servicio en funcionamiento. También aprendimos como administrar, iniciar, eliminar y detener un contendor a través de una serie de comandos, realizamos un ejemplo donde descargamos la imagen httpd, levantamos un servidor web y comprobamos su acceso desde el navegador. Esta guía también nos ayudó entender como desplegar servicios de forma rápida y consistente.